JAVANAISE

Cache d'objets répartis en Java

Documentation technique complète

JVN1 – JVN2 – Extensions – Tests – Guide d'installation et de démonstration

JVN2 - Annotations + Dynamic Proxy

Gestion transparente des verrous grâce à `@JvnRead`, `@JvnWrite` et `JvnProxy`.

JVN1 - Cohérence, verrous et cache local

Gestion explicite de `READ`/`WRITE`, états de verrou et invalidations.

Java RMI - Communication client / coordinateur

Appels distants, callbacks d'invalidation et échange des états sérialisés.

M2 Génie Informatique – UGA

Projet de systèmes et applications répartis – 2026

Table des matières

1 Présentation générale **3**
1.1 Objectifs fonctionnels . . . 3
1.2 Technologies utilisées . . . 3

2 Architecture du système **3**
2.1 Vue d'ensemble . . . 3
2.2 Rôle des classes principales . . . 4
2.3 Structures du coordinateur . . . 4
2.4 Cycle de vie d'un objet partagé . . . 4

3 JVN1 - Gestion explicite des verrous **4**
3.1 États de verrou . . . 5
3.2 `jvnLockRead()` . . . 5
3.3 `jvnLockWrite()` . . . 5
3.4 Correction de course : `writeRequestPending` . . . 5
3.5 `jvnUnLock()` . . . 6
3.6 Invalidations . . . 6
3.7 Test JVN1 . . . 6

4 JVN2 - Transparence par proxy dynamique **6**
4.1 Fichiers ajoutés . . . 6
4.2 Annotations métier . . . 7
4.3 Le proxy comme objet intermédiaire . . . 7
4.4 `JvnProxy.invoke()` . . . 7
4.5 Création du proxy . . . 7
4.6 Différence JVN1 / JVN2 . . . 8

5 Extensions implémentées **8**
5.1 Extension 2 - Pannes clients . . . 8
5.2 Extension 6 - Clients multi-threadés . . . 9
5.3 Extension 1 - Saturation du cache . . . 9
5.3.1 Protocole de flush . . . 9
5.3.2 Politique de victime actuelle . . . 9

6 Tests et validation **10**
6.1 Scripts de démonstration . . . 10
6.2 Pourquoi une barrière de départ ? . . . 10

7 Installation et exécution **10**
7.1 Prérequis . . . 10
7.2 Récupération du projet . . . 11
7.3 Compilation manuelle . . . 11
7.4 Exécution manuelle JVN1 . . . 11
7.5 Exécution par branche . . . 11
7.6 Nettoyage des processus . . . 11

8 Organisation Git et branches **11**

9 Limites et améliorations possibles **12**

10 English installation and usage guide **12**
10.1 Requirements . . . 12
10.2 Clone and compile . . . 12

10.3 Run JVN1 manually 12
10.4 Branches 13
10.5 Recommended demos 13
10.6 Expected behavior 13

A Fichiers principaux **13**

B Aide-mémoire soutenance **14**

C Checklist de démonstration **14**

Références **15**

1. Présentation générale

Javanaise est un service de gestion de cache d'objets Java répartis construit au-dessus de Java RMI. Plusieurs applications peuvent manipuler un même objet logique tout en conservant des copies locales. Le coordinateur maintient la cohérence entre ces copies à l'aide de verrous de lecture et d'écriture.

Le modèle visé est une cohérence de type *entry consistency* : la cohérence est rétablie lorsqu'un client acquiert un verrou. Le projet progresse en deux versions principales, JVN1 puis JVN2, avant l'ajout d'extensions de robustesse et de performance.

Idée centrale

Un objet partagé peut être présent dans plusieurs JVM, mais une seule version cohérente doit être utilisée selon les droits READ/WRITE accordés par le coordinateur.

1.1 Objectifs fonctionnels

— Créer un objet partagé et lui attribuer un identifiant unique.
— Associer un nom symbolique global à un objet partagé.
— Retrouver un objet avec `lookup` depuis une autre JVM.
— Accorder des verrous READ et WRITE en maintenant la cohérence des copies.
— Conserver localement des droits de verrou dans un cache afin d'éviter des appels distants inutiles.
— Permettre au coordinateur d'invalider ou de dégrader les droits détenus par les clients.
— En JVN2, rendre la gestion des verrous transparente grâce aux annotations et aux proxies dynamiques.

1.2 Technologies utilisées

Élément	Rôle
Java RMI	Communication distante entre `JvnServerImpl` et `JvnCoordImpl`, ainsi que callbacks d'invalidation.
Sérialisation Java	Transport de l'état des objets partagés entre JVM.
`synchronized` / `wait` / `notifyAll`	Synchronisation locale et attente pendant une invalidation ou un accès concurrent.
Reflection / Dynamic Proxy	Interception automatique des appels métier dans JVN2.
Git	Séparation des versions `main`, `jvn2` et `extensions`.

2. Architecture du système

2.1 Vue d'ensemble

Application cliente ⟶ JvnObject / JvnProxy ⟶ JvnServerImpl ⟺ JvnCoordImpl

Application métier	Cache local + verrous	RMI	Arbitrage global

2.2 Rôle des classes principales

Classe / interface	Responsabilité
`JvnCoordImpl`	Coordinateur global. Génère les identifiants, enregistre les objets, conserve le dernier état connu, mémorise un writer et un ensemble de readers par objet.
`JvnServerImpl`	Serveur local singleton de chaque application. Fait le lien avec le coordinateur, conserve `localObjects` et reçoit les callbacks d'invalidation.
`JvnObjectImpl`	Objet d'interposition local autour de l'objet métier. Contient `objectState`, `objectId`, `lockState` et la logique de lock/unlock/invalidation.
`JvnRemoteCoord`	Interface RMI exposée par le coordinateur.
`JvnRemoteServer`	Interface RMI permettant au coordinateur de rappeler les clients.
`JvnLocalServer`	API locale utilisée par les applications et les objets JVN.
`JvnException`	Exception fonctionnelle du service JVN.

2.3 Structures du coordinateur

```
private int objectIdCounter;
private Map<String, JvnObject> objectsByName;
private Map<Integer, JvnObject> objectsById;
private Map<Integer, JvnRemoteServer> writers;
private Map<Integer, Set<JvnRemoteServer>> readers;
private Map<Integer, Serializable> objectStates;
```

Structure	Signification
`objectsByName`	nom symbolique → objet JVN ; utilisé pour `lookup`.
`objectsById`	identifiant → objet JVN ; index par ID.
`writers`	ID → client qui possède le droit d'écriture.
`readers`	ID → ensemble des clients qui possèdent un droit de lecture.
`objectStates`	ID → dernier état connu par le coordinateur.

2.4 Cycle de vie d'un objet partagé

1. **Création :** le client demande un nouvel ID au coordinateur puis crée un `JvnObjectImpl` local en état W.
2. **Enregistrement :** `jvnRegisterObject(name, object)` publie le nom et initialise writers/readers/objectState
3. **Lookup :** un autre client demande l'objet par son nom ; la copie reçue est rattachée au serveur local via `setLocalServer()`.
4. **Accès :** le client prend READ ou WRITE, exécute l'opération métier puis libère le verrou.
5. **Invalidation :** si un autre client réclame un droit incompatible, le coordinateur rappelle l'ancien détenteur pour invalider ou dégrader son droit.

3. JVN1 - Gestion explicite des verrous

JVN1 expose directement les primitives de verrouillage. Le programmeur doit prendre le verrou approprié, accéder à l'objet métier via `jvnGetObjectState()`, puis appeler `jvnUnLock()`.

```
jo.jvnLockWrite();
Counter counter = (Counter) jo.jvnGetObjectState();
counter.increment();
jo.jvnUnLock();
```

3.1 États de verrou

État	Signification	Interprétation
NL	No Lock	Aucun droit local.
R	Read	Lecture en cours.
W	Write	Écriture en cours.
RC	Read Cached	Droit de lecture conservé en cache, non utilisé activement.
WC	Write Cached	Droit d'écriture conservé en cache, non utilisé activement.
RWC	Read + Write Cached	Lecture active alors qu'un droit WRITE reste en cache.

3.2 `jvnLockRead()`

La méthode essaie d'abord d'obtenir READ localement. Si aucun droit n'est en cache (NL), elle demande l'état au coordinateur. Si RC ou WC est déjà présent, aucun appel RMI n'est nécessaire.

```
NL     -> demande READ au coordinateur -> R
RC     -> R
WC     -> RWC
```

Pourquoi deux blocs synchronized ?

Le premier protège la décision locale sur `lockState`. L'appel RMI est fait hors du `synchronized` afin qu'une invalidation puisse entrer. Le second bloc installe la réponse du coordinateur sans conflit.

3.3 `jvnLockWrite()`

WRITE est exclusif. Si un droit WC est déjà en cache, le client passe localement à W. En NL ou RC, il demande WRITE au coordinateur.

```
WC      -> W (local)
NL / RC -> demande WRITE au coordinateur -> W
```

3.4 Correction de course : `writeRequestPending`

Un cas critique a été détecté lors du test concurrent : le coordinateur pouvait accorder WRITE, puis envoyer une invalidation avant que le client ait eu le temps d'installer localement `lockState = W`. Le compteur pouvait alors revenir à une ancienne valeur.

```
private boolean writeRequestPending = false;

// avant l'appel RMI
writeRequestPending = true;

// apres installation de W
writeRequestPending = false;
notifyAll();
```

L'invalidation writer attend désormais aussi tant que `writeRequestPending` est vrai. Cette variable représente donc l'état intermédiaire : *« une demande WRITE est en cours »*.

Symptôme avant correction

Avec 3 clients lancés réellement en concurrence, la sortie pouvait faire 1, 2, 3, 4, 5, 6, puis revenir à 3. Après correction, la suite atteint 60 sans retour en arrière.

3.5 jvnUnLock()

```
R   -> RC
W   -> WC
RWC -> WC
notifyAll()
```

Le client ne rend pas forcément immédiatement son droit au coordinateur : il le conserve en cache. C'est ce cache de droits qui réduit les appels RMI.

3.6 Invalidations

Méthode	Comportement
`jvnInvalidateReader()`	Attend la fin d'une lecture active puis passe à NL.
`jvnInvalidateWriter()`	Attend la fin d'une écriture ou d'une demande WRITE en cours, passe à NL et renvoie `objectState`.
`jvnInvalidateWriterForReader()`	Dégrade le droit WRITE en droit de lecture et renvoie l'état courant.

3.7 Test JVN1

Le test de démonstration démarre trois writers et un reader. Chaque writer prend WRITE, incrémente une fois, libère le verrou puis recommence. Une barrière de départ permet de provoquer une concurrence réelle.

```
Client 1 [WRITE] : valeur = 1
Client 2 [WRITE] : valeur = 2
Reader   [READ]  : valeur = 2
Client 3 [WRITE] : valeur = 3
...
Valeur finale attendue : 60
```

Validation

3 writers × 20 incréments = 60. Après la correction `writeRequestPending`, les valeurs observées progressent sans perte de mise à jour.

4. JVN2 - Transparence par proxy dynamique

JVN2 conserve la couche de cohérence de JVN1, mais retire de l'application les appels explicites aux verrous. Le programmeur appelle directement les méthodes métier ; un proxy dynamique intercepte l'appel et choisit automatiquement READ ou WRITE.

4.1 Fichiers ajoutés

Fichier	Rôle
`JvnRead.java`	Annotation runtime marquant une méthode de lecture.
`JvnWrite.java`	Annotation runtime marquant une méthode d'écriture.
`JvnProxy.java`	`InvocationHandler` : intercepte les appels, prend le verrou, appelle l'objet réel puis `unlock`.
`SentenceItf.java`	Interface métier annotée pour `Sentence`.
`CounterItf.java`	Interface métier annotée pour `Counter`.

4.2 Annotations métier

```
public interface CounterItf {
    @JvnWrite
    void increment();

    @JvnRead
    int getValue();
}
```

Les annotations ne prennent aucun verrou par elles-mêmes. Elles servent uniquement de métadonnées lisibles par le proxy au moment de l'exécution.

4.3 Le proxy comme objet intermédiaire

Définition simple

Le proxy est un objet placé entre l'application et le vrai objet métier. L'application pense appeler `Counter`, mais l'appel passe d'abord dans `JvnProxy.invoke()`.

counter.increment() → Proxy → @JvnWrite → lock WRITE → Counter.increment() → unlock

4.4 `JvnProxy.invoke()`

```
if (method.isAnnotationPresent(JvnRead.class)) {
    jvnObject.jvnLockRead();
} else if (method.isAnnotationPresent(JvnWrite.class)) {
    jvnObject.jvnLockWrite();
}

try {
    Object realObject = jvnObject.jvnGetObjectState();
    return method.invoke(realObject, args);
} finally {
    if (method.isAnnotationPresent(JvnRead.class)
            || method.isAnnotationPresent(JvnWrite.class)) {
        jvnObject.jvnUnLock();
    }
}
```

Le `finally` est important : le verrou est libéré même si la méthode métier lève une exception.

4.5 Création du proxy

```
CounterItf counter =
    (CounterItf) JvnProxy.newInstance(jo);
```

`Proxy.newProxyInstance()` crée à l'exécution un objet qui implémente les mêmes interfaces que l'objet réel. Le programme peut donc manipuler `CounterItf` sans connaître directement `JvnObject`.

4.6 Différence JVN1 / JVN2

JVN1	JVN2
Le code appelle `jvnLockRead/Write` et `jvnUnLock`.	Le code appelle uniquement les méthodes métier.
L'application connaît `JvnObject`.	L'application manipule une interface métier.
Risque d'oublier un verrou ou un unlock.	Le proxy centralise la logique de verrouillage.
Plus explicite, moins transparent.	Plus transparent et plus proche du code métier.

5. Extensions implémentées

Le sujet propose sept extensions. Trois ont été implémentées dans la branche `extensions`. Leur numéro officiel est conservé ci-dessous.

Extension	Sujet	État
1	Saturation du cache client	✓ Implémentée
2	Traitement des pannes client	✓ Implémentée
3	Panne du coordinateur	× Non implémentée
4	Coordinateur décentralisé	× Non implémentée
5	Invocations transactionnelles ACID	× Non implémentée
6	Clients multi-threadés	✓ Implémentée
7	Référence JVN vers un autre objet JVN	× Non implémentée

5.1 Extension 2 - Pannes clients

Le coordinateur effectue des callbacks RMI vers les clients lors des invalidations. Si une JVM client meurt, ces appels lèvent `RemoteException`. L'extension capture ces exceptions et retire le client défaillant des structures de coordination au lieu de faire échouer tout le service.

```
try {
    Serializable state = writer.jvnInvalidateWriter(joi);
    objectStates.put(joi, state);
} catch (RemoteException e) {
    writers.remove(joi);
}
```

Cas	Comportement
Reader en panne	L'invalidation échoue, le coordinateur affiche la détection de panne puis vide/actualise la liste des readers.
Writer en panne	Le coordinateur retire le writer et repart du dernier `objectState` qu'il connaissait.
Limite	Si le writer avait une modification uniquement locale et meurt avant de la renvoyer, cette dernière modification est perdue.

Test observé

Un `WriterCrashTest` passe localement 0 → 1, puis est tué avec `kill -9`. Le client suivant repart de 0 et atteint 10000 : le +1 local est perdu, mais le service continue.

5.2 Extension 6 - Clients multi-threadés

Un même processus client peut contenir plusieurs threads utilisant le même `JvnObject`. Le simple état W/R n'est pas suffisant pour distinguer plusieurs utilisateurs locaux. Deux variables ont donc été ajoutées.

```
private int activeReaders = 0;
private boolean activeWriter = false;
```

Cas	Règle locale
READ + READ	Autorisé : `activeReaders` est incrémenté.
WRITE + READ	Interdit simultanément : le second thread attend.
WRITE + WRITE	Interdit simultanément : un seul `activeWriter`.
Fin d'accès	`jvnUnLock()` décrémente `activeReaders` ou remet `activeWriter=false`, puis `notifyAll()`.

Résultats

10 threads × 1000 WRITE donnent 10000. Un test mixte 5 writers × 1000 + 5 readers × 1000 donne 5000, sans perte d'écriture.

5.3 Extension 1 - Saturation du cache

`JvnServerImpl` conserve les objets locaux dans `localObjects`. Sans limite, ce cache peut grossir indéfiniment. La branche `extensions` fixe une taille maximale de 3 objets et déclenche un flush lorsqu'un quatrième objet est ajouté.

```
private static final int MAX_CACHE_SIZE = 3;

if (localObjects.size() > MAX_CACHE_SIZE) {
    jvnFlushObject(victimId);
}
```

5.3.1 Protocole de flush

Méthode	Rôle
`jvnPrepareFlush()`	Bloque de nouveaux accès et attend la fin des readers/writer locaux. Renvoie l'état courant.
`jvnFlushObject()` côté client	Prévient le coordinateur, confirme le flush puis retire l'objet de `localObjects`.
`jvnFlushObject()` côté coordinateur	Retire le client des readers ; s'il était writer, sauvegarde son état et retire son droit WRITE.
`jvnConfirmFlush()`	Marque l'objet comme `evicted`, remet le lock à NL et termine le flush.
`jvnCancelFlush()`	Annule le flush si l'appel distant échoue et rend l'objet à nouveau utilisable.

Important

Un flush ne supprime pas l'objet global. Il le retire seulement du cache local du client. Un futur `lookup` peut recharger l'objet avec son dernier état connu.

5.3.2 Politique de victime actuelle

La victime est le premier identifiant trouvé dans la `HashMap`, différent de l'objet protégé qui vient d'être créé ou chargé. Ce choix fonctionne pour démontrer la saturation, mais ce n'est ni un vrai

FIFO ni un LRU. Une amélioration naturelle serait d'utiliser `LinkedHashMap` ou une structure dédiée de recency.

6. Tests et validation

Les tests ont été construits pour provoquer de la concurrence réelle, pas seulement des appels séquentiels. Les scripts nettoient les anciens processus RMI, compilent, démarrent le coordinateur, créent `COUNTER`, lancent les clients puis nettoient l'environnement.

Test	Scénario	But	Attendu	Résultat
JVN1 burst	3 writers x20 + reader	Cohérence WRITE/READ et invalidations	60	60 après correction
JVN2 burst	3 clients x10000 via proxy	Verrous automatiques `@JvnWrite`	30000	30000
Panne reader	READ puis `kill -9`	Tolérer un lecteur mort	Service continue	OK
Panne writer	WRITE local puis `kill -9`	Tolérer un writer mort	Service continue	OK, dernière modif perdue
Multi-thread WRITE	10 threads x1000	Exclusion locale des writers	10000	10000
Multi-thread mixte	5 writers + 5 readers	READ/WRITE dans une JVM	5000	5000
Flush manuel	Modifier, flush, lookup	Conserver l'état après éviction	Valeur conservée	OK
Saturation auto	Créer 4 objets, max=3	Déclencher un flush	1 objet flushé	OK

6.1 Scripts de démonstration

Script	Démonstration
`run_jvn1_tests.sh`	Compile et lance le scénario JVN1 concurrent.
`run_jvn2_tests.sh`	Compile et lance JVN2 avec 3 writers + 1 reader ; le proxy gère les verrous.
`run_client_failure_tests.sh`	Tue volontairement `ReaderCrashTest` puis `WriterCrashTest` et vérifie que le système continue.
`run_multithread_tests.sh`	Exécute `MultiThreadCounterTest` et `MultiThreadReadWriteTest`.
`run_cache_tests.sh`	Exécute le flush manuel puis la saturation automatique du cache.

6.2 Pourquoi une barrière de départ ?

Lancer trois commandes avec `&` ne garantit pas qu'elles commencent réellement leur boucle au même instant. Les scripts de burst utilisent un fichier signal comme `/tmp/jvn_start` : chaque client attend son apparition, puis tous sont libérés presque simultanément. Cela rend le test beaucoup plus agressif et a permis de révéler la course corrigée avec `writeRequestPending`.

7. Installation et exécution

7.1 Prérequis

— JDK récent avec `java` et `javac` disponibles dans `PATH`.
— Git pour récupérer le dépôt et changer de branche.

— Linux / WSL recommandé pour exécuter les scripts Bash.
— Port RMI 1099 disponible localement.

7.2 Récupération du projet

```
git clone https://github.com/Takissam21/Javanaise.git
cd Javanaise
```

7.3 Compilation manuelle

```
rm -rf bin
mkdir bin
javac -d bin src/jvn/*.java src/irc/*.java
```

7.4 Exécution manuelle JVN1

```
# Terminal 1
java -cp bin jvn.JvnCoordImpl

# Terminal 2
java -cp bin irc.CounterInit

# Terminaux suivants
java -cp bin irc.CounterBurstTest "Client 1"
java -cp bin irc.CounterReaderTest "Reader"
```

7.5 Exécution par branche

Branche	Commande / objectif
`main`	`git switch main` puis `./run_jvn1_tests.sh`.
`jvn2`	`git switch jvn2` puis `./run_jvn2_tests.sh`.
`extensions`	`git switch extensions` puis lancer les scripts de panne, multi-thread et cache.

7.6 Nettoyage des processus

```
jps
pkill -f "jvn.JvnCoordImpl"
pkill -f "irc.CounterInit"
pkill -f "irc.CounterBurstTest"
```

Les anciennes JVM RMI peuvent provoquer des `Connection refused` ou des callbacks vers des processus morts. Les scripts effectuent donc un nettoyage avant chaque scénario.

8. Organisation Git et branches

Branche	Contenu
`main`	Version JVN1, gestion explicite des verrous, correction `writeRequestPending`, tests burst READ/WRITE.
`jvn2`	JVN2 avec `JvnProxy`, `@JvnRead`, `@JvnWrite` et interfaces métier annotées.
`extensions`	JVN2 enrichi avec panne client, multi-thread local et saturation/flush de cache.

Cette séparation permet de démontrer l'évolution du projet sans mélanger les concepts : d'abord la cohérence de base, ensuite la transparence, enfin les extensions.

```
git switch main
git switch jvn2
git switch extensions

git status
git log --oneline --decorate -10
```

9. Limites et améliorations possibles

— **Panne writer :** une modification non encore récupérée par le coordinateur peut être perdue.
— **Panne coordinateur :** non traitée ; le coordinateur reste un point unique de défaillance.
— **Cache :** politique de victime actuelle non déterministe (`HashMap`), pas de LRU/FIFO réel.
— **Persistance :** aucun journal durable pour reconstruire l'état global après redémarrage du coordinateur.
— **Transactions :** pas de groupe atomique de plusieurs opérations métier.
— **Références imbriquées :** un objet JVN ne gère pas spécialement une référence vers un autre objet JVN.
— **Validation :** les tests sont solides pour les scénarios ciblés mais ne constituent pas une preuve formelle de sûreté pour toutes les interleavings possibles.

Améliorations prioritaires

1) vraie politique LRU ; 2) persistance du coordinateur ; 3) tests de charge plus longs avec seeds reproductibles ; 4) métriques RMI et temps d'attente ; 5) extension 3 de reprise du coordinateur.

10. English installation and usage guide

This section is intentionally written in English so it can be reused as the installation / usage manual requested with the project deliverables.

10.1 Requirements

— Java Development Kit (JDK) with `java` and `javac` in PATH.
— Git.
— Linux or WSL for the provided Bash scripts.
— TCP port 1099 available for the local RMI registry.

10.2 Clone and compile

```
git clone https://github.com/Takissam21/Javanaise.git
cd Javanaise
rm -rf bin && mkdir bin
javac -d bin src/jvn/*.java src/irc/*.java
```

10.3 Run JVN1 manually

```
# Terminal 1: coordinator
java -cp bin jvn.JvnCoordImpl
```

```
# Terminal 2: create/register COUNTER
java -cp bin irc.CounterInit

# Other terminals: clients
java -cp bin irc.CounterBurstTest "Client 1"
java -cp bin irc.CounterReaderTest "Reader"
```

10.4 Branches

Branch	Purpose
`main`	JVN1 - explicit lock management.
`jvn2`	JVN2 - annotations and Java Dynamic Proxy.
`extensions`	Client failures, multi-threaded clients and client-cache saturation.

10.5 Recommended demos

```
git switch main
./run_jvn1_tests.sh

git switch jvn2
./run_jvn2_tests.sh

git switch extensions
./run_client_failure_tests.sh
./run_multithread_tests.sh
./run_cache_tests.sh
```

10.6 Expected behavior

— Concurrent writers must not lose increments.
— Readers may observe intermediate values while writers are still running, but they must never corrupt the counter.
— A dead client must be detected through a failed RMI callback and removed from coordinator ownership structures.
— Multiple local readers may coexist, while a local writer remains exclusive.
— When the local cache exceeds its maximum size, one cached object is flushed and can later be reloaded through lookup.

A. Fichiers principaux

Fichier	Rôle
`src/jvn/JvnCoordImpl.java`	Coordinateur et arbitrage global des verrous.
`src/jvn/JvnServerImpl.java`	Serveur local, cache, appels au coordinateur, callbacks.
`src/jvn/JvnObjectImpl.java`	État local, verrous, `wait/notify`, invalidations, extensions.
`src/jvn/JvnObject.java`	Contrat de l'objet d'interposition.
`src/jvn/JvnRemoteCoord.java`	API distante du coordinateur.
`src/jvn/JvnRemoteServer.java`	API de callback vers les clients.
`src/jvn/JvnProxy.java`	Interception JVN2.
`src/jvn/JvnRead.java / JvnWrite.java`	Annotations de mode d'accès.

Fichier	Rôle
`src/irc/Irc.java / Sentence.java`	Application de chat fournie/adaptée.
`src/irc/Counter*.java`	Objet compteur, init et tests de concurrence.
`src/irc/ReaderCrashTest.java`	Client volontairement bloqué après READ pour test de panne.
`src/irc/WriterCrashTest.java`	Client volontairement bloqué après WRITE local pour test de panne.
`src/irc/MultiThreadCounterTest.java`	Test 10 writers locaux.
`src/irc/MultiThreadReadWriteTest.java`	Test mixte local readers/writers.
`src/irc/CacheFlushTest.java`	Test de flush manuel.
`src/irc/CacheSaturationTest.java`	Test d'éviction automatique quand la taille maximale du cache est dépassée.

B. Aide-mémoire soutenance

Question	Réponse courte
Pourquoi JVN ?	Partager des objets Java entre JVM tout en gardant des copies locales cohérentes.
Rôle du coordinateur ?	Savoir qui lit, qui écrit, conserver l'état courant et invalider les droits incompatibles.
Pourquoi RC/WC ?	Conserver des droits en cache pour éviter un appel RMI à chaque accès.
Pourquoi `synchronized` ?	Protéger `lockState/objectState` contre des appels concurrents application + callbacks RMI.
Pourquoi appel RMI hors `synchronized` ?	Éviter de bloquer une invalidation et donc un interblocage.
Pourquoi `writeRequestPending` ?	Protéger la fenêtre entre la demande WRITE et l'installation locale de W.
JVN2 apporte quoi ?	Le proxy prend et libère automatiquement les verrous selon `@JvnRead/@JvnWrite`.
Panne client ?	Catch `RemoteException`, retirer le client mort, continuer avec le dernier état connu.
Multi-thread ?	Plusieurs READ locaux autorisés, WRITE exclusif.
Flush cache ?	Retirer un objet du cache local sans le supprimer globalement.

C. Checklist de démonstration

— Vérifier la branche active avec `git branch -show-current`.
— Nettoyer les anciennes JVM avant chaque scénario.
— Compiler avec `javac` avant la démonstration.
— JVN1 : montrer 3 writers + 1 reader et une valeur finale cohérente.
— JVN2 : insister sur l'absence de `jvnLockRead/jvnLockWrite` dans le code métier.
— Panne client : montrer le message de détection après `kill -9`.
— Multi-thread : comparer valeur attendue et valeur obtenue.
— Cache : montrer le flush manuel puis le message de saturation automatique.
— À l'oral, distinguer `lockState` (droit/verrou) et `objectState` (objet métier).
— Terminer en rappelant les limites : writer mort peut perdre sa dernière modification locale ; cache non-LRU ; coordinateur non tolérant aux pannes.

Références

— F. Boyer, *JAVANAISE : un cache d'objets répartis en Java*, UE Systèmes et Applications Répartis, M2GI / IM2AG, UGA.
— Supports complémentaires : Java Annotations et Java Dynamic Proxies fournis avec le projet.
— Dépôt du projet : `https://github.com/Takissam21/Javanaise` (branches `main`, `jvn2`, `extensions`).